

LINKED LIST

ANIMATIONS

WEEK # 6

LINKED LIST

```
class linked_list
{
    private:
        node *head, *last;
    public:
        list()
        {
            head=NULL;
            last=NULL;
        }
};
```

INSERTION – END

```
void createnode(int value)
```

```
{
```

```
    node *temp=new node;
```

```
    temp->data=value;
```

```
    temp->next=NULL;
```

```
    if(head==NULL)
```



True

```
    {
```

```
        head=temp;
```

```
        last=temp;
```

```
    }
```

```
    else
```

```
    {
```

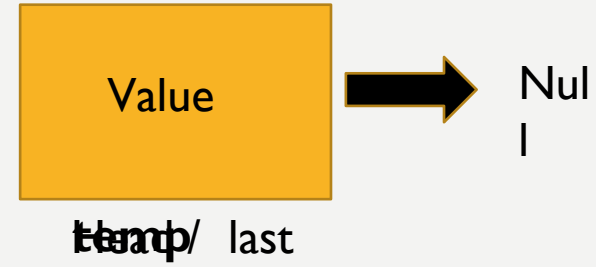
```
        last->next=temp;
```

```
        last=temp;
```

```
    }
```

```
}
```

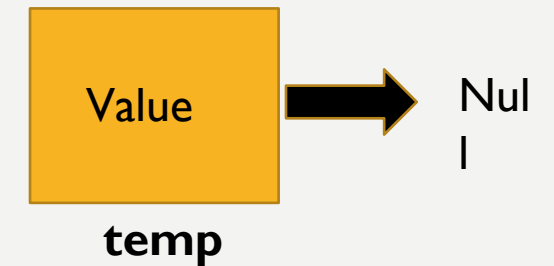
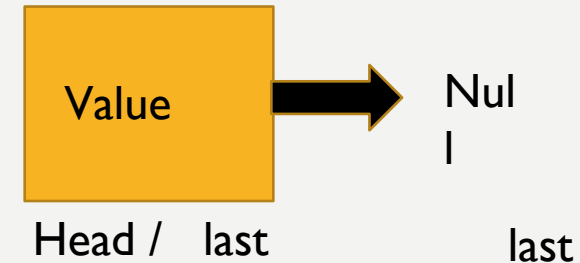
**Will Not
Execute**



```

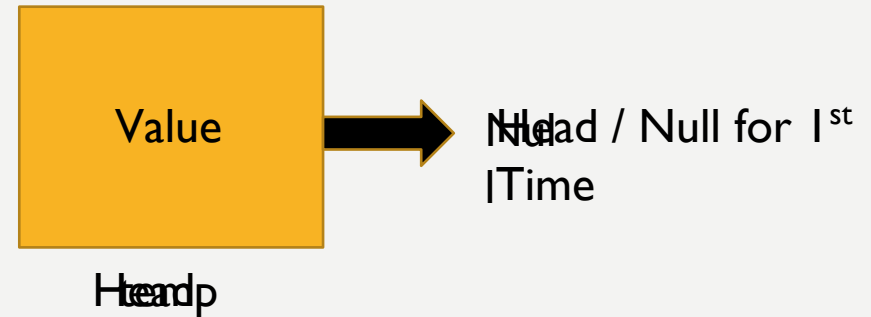
void createnode(int value)
{
    node *temp=new node;
    temp->data=value;
    temp->next=NULL;
    if(head==NULL)  —————→ False as head has a value
    {
        head=temp;
        last=temp;
    }
    else
    {
        last->next=temp;
        last=temp;
    }
}

```



INSERTION - BEGINNING

```
void insert_start(int value)
{
    node *temp=new node;
    temp->data=value;
    temp->next=head;
    head=temp;
}
```



INSERT TO SOME POSITION

Pos = 3

```
void insert_position(int pos, int value)
```

```
{
```

```
    node *pre=new node;
```

```
    node *cur=new node;
```

```
    node *temp=new node;
```

```
    cur=head;
```

```
    for(int i=1;i<pos;i++)
```

```
    {
```

```
        pre=cur;
```

```
        cur=cur->next;
```

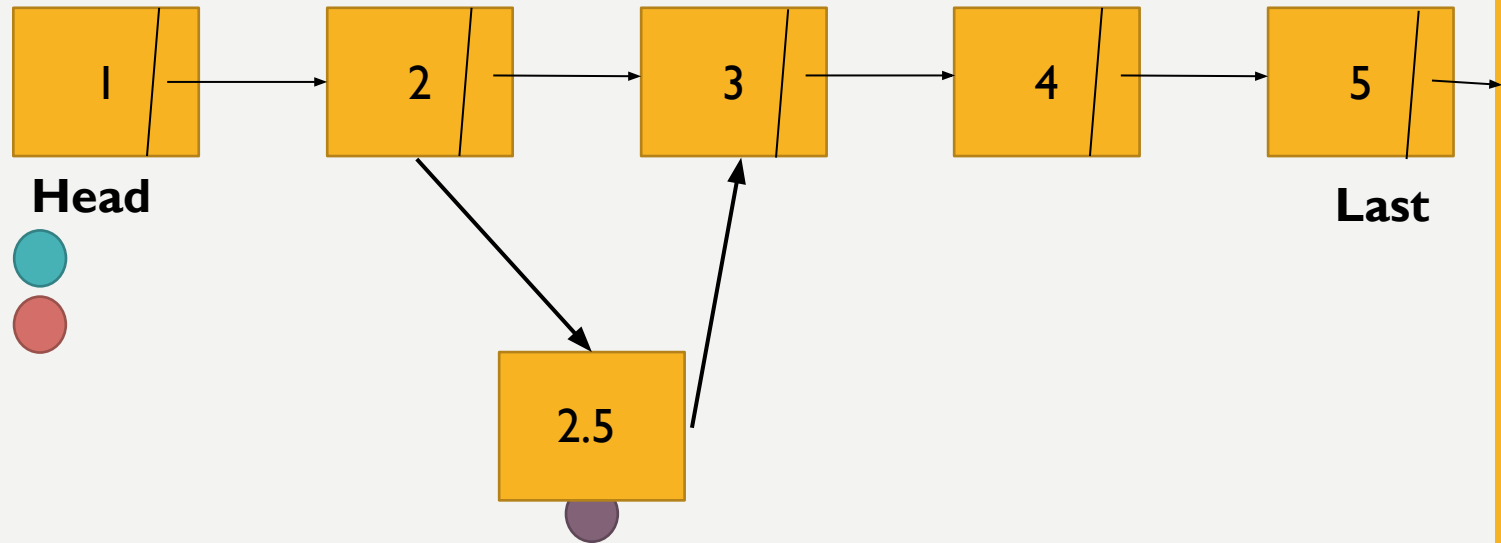
```
    }
```

```
    temp->data=value;
```

```
    pre->next=temp;
```

```
    temp->next=cur;
```

```
}
```



DISPLAY

```
void display()
{
    node *temp=new node;
    temp=head;
    while(temp!=NULL)
    {
        cout<<temp->data<<"\t";
        temp=temp->next;
    }
}
```

DELETE - START

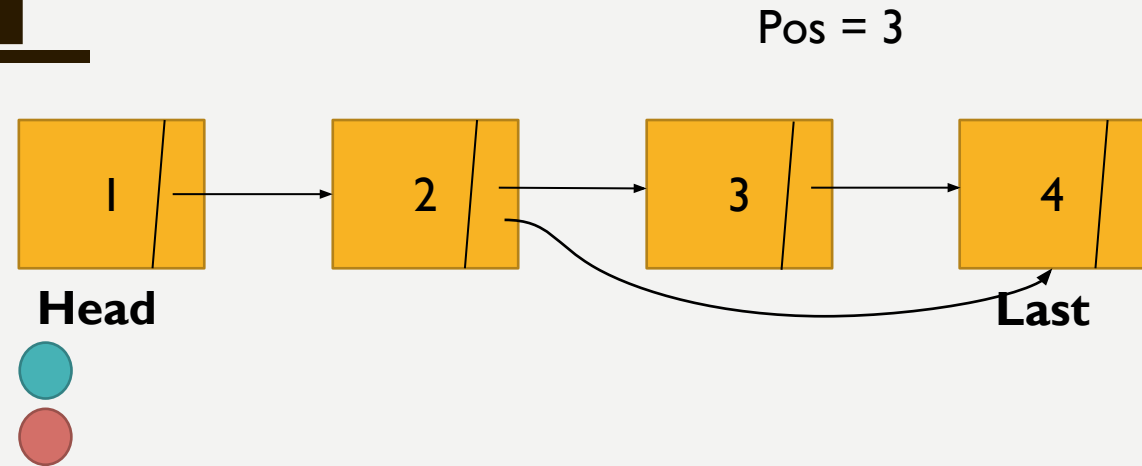
```
void delete_first()
{
    node *temp=new node;
    temp=head;
    head=head->next;
    delete temp;
}
```


DELETE - END

```
void delete_last()
{
    node *current=new node;
    node *previous=new node;
    current=head;
    while(current->next!=NULL)
    {
        previous=current;
        current=current->next;
    }
    last=previous;
    previous->next=NULL;
    delete current;
}
```

DELETE - POSITION

```
void delete_position(int pos)
{
    node *current=new node;
    node *previous=new node;
    current=head;
    for(int i=1;i<pos;i++)
    {
        previous=current;
        current=current->next;
    }
    previous->next=current->next;
    delete current;
}
```



DOUBLY LINKED-LIST

```
void insertEnd(struct Node** start, int value)
{
    if (*start == NULL)
    {
        Node* new_node = new Node;
        new_node->data = value;
        new_node->next = new_node->prev = new_node;
        *start = new_node;
        return;
    }
    Node *last = (*start)->prev;
    struct Node* new_node = new Node;
    new_node->data = value;
    new_node->next = *start;
    (*start)->prev = new_node;
    new_node->prev = last;
    last->next = new_node;
}
```